

Architecture Justification

Student Startup Crowdfunding Platform

Architecture Selection

We chose Microservices Architecture for the Student Startup Crowdfunding Platform.

Reason 1 – Separate Services

The system has different functions such as user authentication, campaign management, pledges and rewards, milestone management, and payment processing. These functions also change on different schedules and are owned by different concerns: milestone approvals are reviewed by the faculty committee and updated only when a milestone is verified, while pledge and reward logic changes far more often as new reward tiers or campaign rules are introduced. Microservices allows each function to be built, updated, and deployed as an independent service, so a change to the milestone approval process does not require redeploying the pledge or authentication services.

Reason 2 – Independent Scaling

Different services may receive different amounts of traffic. For example, the Pledge and Payment services may receive many requests when backers are making pledges, especially near a campaign deadline or when a campaign goes viral. Microservices allows individual services to be scaled independently, so only the services under load need additional resources.

Security Advantage

Payment processing can be kept as a separate service. This helps isolate sensitive payment information and allows security controls, such as PCI-DSS compliant handling and tokenized authorizations, to be applied specifically to the payment service without affecting the rest of the platform.

Performance Benefit

Because each service runs independently, a slow or heavily loaded service does not degrade the performance of the others. For example, if the Payment service is temporarily slow while communicating with an external payment gateway, students and backers can still browse campaigns and view milestone updates without delay, since Campaign Management and Milestone Management run as separate services. Similarly, a surge of pledge activity on a popular campaign does not slow down user login or authentication requests for the rest of the platform. This fault isolation, rather than scaling alone, is the main performance benefit of the microservices approach.